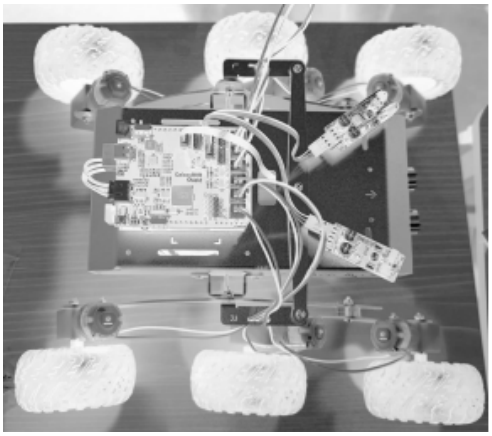
Proyecto de Innovación Educativa

Bachillerato de investigación

IES Santa Catalina de Alejandría
Jaén

Diseño, desarrollo y construcción de un vehículo de exploración científica controlado a distancia tipo "Rover".



Autoría: Romero Anguita, Francisco José
Materia: Tecnología e Informática
Centro: IES Santa Catalina de Alejandría
Localidad: Jaén
Tutor interno: Dª María del Rocío Escabias Machuca
Tutor externo: D. Jesús Salas Jiménez

10 / 2025

Índice

1. Abstract **5**

2. Introducción **6**

3. Estado de la cuestión (antecedentes/marco teórico) **8**

4. Procedimiento (metodología de investigación) **11**
4.1. Revisión Bibliográfica 12
4.2. Búsqueda de componentes y recursos 13
4.3. Montaje del vehículo y diseño de circuitos 15
4.4. Programación 20
4.4.1. Movimiento teledirigido y GUI 20
4.4.2. Estación meteorológica y de gases 22
4.4.3. Estación óptica 25
4.4.4. Localización RSSI 26
4.4.5. Video transmisión en directo 27
4.4.6. Comunicación a través de WiFi 28
4.5. MATLAB 29

5. Fuentes, materiales y métodos **31**
5.1. Materiales 31
5.2. Software 31
5.3. Metodología 32

6. Resultados **33**
6.1. MATLAB 33
6.1.1. Principal 33
6.1.2. Control de motores 34
6.1.3. Sensor BME-688: Estación Meteorológica 35
6.1.4. Sensor APDS-9960: Estación Óptica 37
6.1.5. Distancia WiFi y cámara 39
6.2. Resultados 40
6.2.1. Prueba 1: Jaén (20/12/2025) 40
6.2.2. Prueba 2: Almuñécar (22/12/2025) 41
6.3. RSSI 42
6.4. Cámara 44

7. Problemas **45**
7.1. Problemas físicos 45
7.2. Problemas de conexión 46
7.3. Problemas de impresión 47

8. Conclusión **48**

9. Bibliografía **49**

10.Anexo **49**

Resumen

El proyecto actual tiene como objetivo desarrollar un trabajo lo más transversal posible, permitiendo la aplicación de competencias STEAM (Ciencia, Tecnología, Ingeniería, Artes y Matemáticas). Esto combina áreas y materias de la Educación Secundaria Obligatoria (ESO) y el Bachillerato, como Tecnología, Robótica, Informática y Programación, y Física, comprobando su sinergia y multifuncionalidad a través del «diseño, construcción, desarrollo y programación de un vehículo de exploración científica tipo "Rover" controlado de forma remota».

Se establecen una serie de objetivos multidisciplinares clave, centrados en dotar al vehículo de exploración de capacidades avanzadas. Entre ellos destacan el movimiento controlado a distancia desde una estación de control, la detección de objetos y la integración de diversas estaciones de monitorización: una estación meteorológica, una estación óptica y una estación de gases. El proyecto también persigue la geolocalización GPS del vehículo, la capacidad de transmisión de vídeo en tiempo real y una comunicación robusta mediante WiFi.

La metodología seguida se basa principalmente en la aplicación práctica. Comienza con una revisión bibliográfica de conceptos fundamentales de física atmosférica, robótica y programación, estableciendo las bases teóricas necesarias. Posteriormente, se trabaja en el diseño y desarrollo de las distintas partes, lo que conduce a la búsqueda y estimación presupuestaria de los componentes esenciales del vehículo, culminando en la adquisición de los componentes y materiales necesarios, así como de las herramientas y máquinas utilizadas en el proceso.

El trabajo continúa profundizando en el diseño del esquema del circuito digital y en otras tareas paralelas de ensamblaje del vehículo. El siguiente paso crítico es la planificación y el desarrollo del programa, el "núcleo" de control de todos los sensores y módulos, así como de la comunicación con MATLAB. Un ejemplo claro de esta implementación es la estación meteorológica, donde la interconexión entre una Raspberry Pi Zero, el sensor BME-688, el código Arduino y la funcionalidad WiFi de la Raspberry permite obtener en MATLAB datos precisos de temperatura, humedad, presión y altitud. Para finalizar esta fase inicial, se crea un modelo 3D que incorpora un mástil para la futura integración de elementos adicionales.

A pesar de los avances y del buen ritmo del proyecto, su desarrollo no está exento de retos, dificultades y obstáculos que superar en cada etapa. Algunos problemas recurrentes incluyen la adquisición de componentes, la rotura de piezas durante el montaje y dificultades significativas con determinados programas informáticos, como el software de diseño de circuitos o la propia aplicación de MATLAB.

1. Abstract

The current project aims to develop a work that is as cross-curricular as possible, allowing for the application of STEAM (Science, Technology, Engineering, Arts, and Mathematics) skills. This combines areas and subjects from compulsory secondary education (ESO) and Baccalaureate such as Technology, Robotics, Computing and Programming, and Physics, checking their synergy and multifunctionality through the "Design, construction, development, and programming of a remote-controlled scientific exploration vehicle of the 'Rover' type."

A series of key multidisciplinary objectives are set, focused on equipping the exploration vehicle with advanced capabilities. Among these, the highlights are remote-controlled movement from a control station, object detection, and the integration of various monitoring stations: a weather station, an optical station, and a gas station. The project also seeks GPS geolocation of the vehicle, the capacity for live video transmission, and robust communication via WiFi.

The methodology followed is based mainly on practical application. It begins with a bibliographic review of fundamental concepts in atmospheric physics, robotics, and programming, establishing the necessary theoretical bases. Subsequently, work is done on the design and development of the different parts, which leads to the search for and budgetary estimation of the essential components for the vehicle, culminating in the acquisition of necessary components and materials as well as tools and machines used in the process.

The work continues by delving into the design of the digital circuit schematic and other parallel tasks of vehicle assembly. The next critical step is the planning and development of the program, the control "core" for all sensors and modules, as well as communication with MATLAB. A clear example of this implementation is the weather station, where the interconnection between a Raspberry Pi Zero, the BME-688 sensor, Arduino code, and the Raspberry's WiFi functionality allows precise data on temperature, humidity, pressure, and altitude to be obtained in MATLAB. To finalize this initial phase, a 3D model is created to incorporate a mast for the future integration of additional elements.

Despite the progress and good pace of the project, its development is not free of challenges, difficulties, and obstacles to overcome at every step. Some recurrent problems include the acquisition of components, the breakage of parts during assembly, and significant difficulties with specific software such as the circuit design software or the MATLAB application itself.

2. Introducción

Un vehículo de tipo "rover", también llamado astromóvil, es un vehículo diseñado para la exploración e investigación de entornos hostiles en los que el ser humano difícilmente podría sobrevivir sin un soporte vital y una logística compleja. Este tipo de sistemas robóticos son una alternativa, mucho más barata y exenta de riesgo, al envío de humanos a misiones científicas y/o de exploración. Además, los vehículos robotizados de esta clase destacan por su resistencia, autonomía y capacidad de adaptación a distintos escenarios y situaciones lo que los convierte en unos "científicos pioneros" excepcionales.

El presente trabajo muestra el diseño, construcción, programación y posterior prueba de funcionamiento de un vehículo rover controlado a distancia pero con cierta autonomía en algunos movimientos. El dispositivo porta un laboratorio de investigación automatizado con capacidad para estudiar el entorno próximo que lo rodea. Asimismo, el vehículo posee transmisión de vídeo en directo y capacidad (ligera) de reconocimiento espacial. La comunicación entre el operador que lo controla y el propio rover se lleva a cabo mediante protocolo WiFi. Las capacidades proyectadas y posteriormente demostradas mediante ensayos y pruebas de campo para este vehículo rover son las siguientes:

1. Movimiento teledirigido desde el puesto de control.

2. Reconocimiento espacial básico y detección de objetos.

3. Estación meteorológica:

   a) Medición de la temperatura del aire

   b) Medición de la humedad del aire.

   c) Medición de la presión atmosférica.

   d) Medición de la altura barométrica.

4. Estación óptica y de gases:

   a) Medición de distintas componentes espectrales de la luz visible. Medición de la intensidad lumínica.

   b) Medición de la calidad del aire circundante.

5. Localización aproximada del vehículo por RSSI[1].

[1] RSSI: "Received Signal Strength Indicator" (Indicador de Intensidad de Señal Recibida), una medida de la potencia de una señal de radiofrecuencia que un receptor capta.

6. Transmisión de video en tiempo real y comunicación bidireccional mediante WiFi.

3. Estado de la cuestión (antecedentes/marco teórico)

Un rover de exploración, como el desarrollado en este proyecto, es un sistema mecatrónico complejo que integra múltiples disciplinas, incluyendo la robótica móvil, la sensórica, la ingeniería de software y las telecomunicaciones. Por lo tanto, este proyecto no se limita a la investigación en un solo campo, sino que establece los principios fundamentales de cada dominio que, al interconectarse, dan funcionalidad al sistema completo.

El diseño del rover se fundamenta en una arquitectura de control distribuido, un modelo que divide las tareas de procesamiento entre múltiples unidades para optimizar el rendimiento, la fiabilidad y la respuesta en tiempo real.

- Ordenador principal: La Raspberry Pi Zero actúa como "cerebro" del rover. Al ejecutar un sistema operativo completo (Linux), se encarga de las tareas de alta complejidad computacional:
  - Gestión de Red: Creación de un punto de acceso WiFi para la teleoperación.
  - Interfaz de Usuario (GUI): Ejecución de la aplicación de control.
  - Telemetría y Control Remoto: Gestión del servidor SSH y los scripts de transmisión de datos.
  - Procesamiento de Vídeo: Codificación y transmisión del flujo de vídeo de la RPiCam.
- Microcontrolador: El Arduino Mega actúa como un "sistema nervioso". Su función es dirigir en tiempo real las tareas que no pueden ser interrumpidas por el cerebro:
  - Control de Actuadores: Generación precisa de señales para motores y servos.
  - Lectura de Sensores: Adquisición de datos brutos de los sensores BME-688, APDS-9960 y ultrasonidos.
- Protocolo de Comunicación Maestro-Esclavo: La coherencia del sistema se logra mediante un protocolo de comunicación serial asíncrono. En esta arquitectura, la Raspberry Pi actúa como Maestro, enviando comandos de un solo byte (ej. 'w', 'e', 'l'), y el Arduino actúa como Esclavo, escuchando y ejecutando dichos comandos.

La locomoción y la interacción física del rover se basan en los siguientes principios de actuación electromecánica.

- Motores de Corriente Continua (CC): La tracción (6WD) se logra mediante motores de CC, que operan bajo el principio de la Ley de Lorentz para convertir la energía eléctrica en movimiento rotacional.

- Control de Velocidad por PWM (Modulación por Ancho de Pulso): Para lograr un control de velocidad variable (y no meramente binario), se emplea la técnica de PWM. Este principio consiste en variar el ciclo de trabajo (el porcentaje de tiempo que la señal está en alto) de un pulso digital. Aunque el voltaje es constante, la potencia promedio entregada al motor varía, permitiendo un control fino de la velocidad. Esto se implementa en el código mediante la librería SoftPWM.

- Servomotor: A diferencia de los motores CC, el servomotor es un actuador de lazo cerrado. Utiliza una señal PWM no para controlar la velocidad, sino para definir una posición angular específica. Un circuito de control interno compara la posición deseada con la posición real (medida por un potenciómetro) y ajusta el motor hasta que el error es cero. En este proyecto, se utiliza para orientar la videocámara.

El rover obtiene conocimiento de su entorno mediante un conjunto de sensores. El fundamento teórico no solo radica en la sensórica en sí, sino en el acondicionamiento digital de la señal para garantizar datos fiables.

- Detección de Gases por Resistencia de Óxido Metálico: El sensor BME-688 (una evolución del BME-680) mide la calidad del aire basándose en el principio de Resistencia de Óxido Metálico (MOx). Un elemento calefactor eleva la temperatura de una capa de óxido metálico. Los Compuestos Orgánicos Volátiles (VOCs) reaccionan con esta superficie, alterando su resistencia eléctrica. El microcontrolador mide esta resistencia para estimar la concentración de gases.

- Medición de Distancia por Tiempo de Vuelo (ToF): El rover emplea dos métodos basados en el principio de "Tiempo de Vuelo" (Time of Flight):

- Ultrasonido: Mide el tiempo que tarda un pulso de sonido en viajar al objeto, rebotar y regresar.

- LIDAR: Mide el tiempo que tarda un pulso de luz (láser) en hacer el mismo recorrido. Al ser la luz mucho más rápida y colimada, el LIDAR ofrece una precisión y frecuencia de muestreo superiores.

- Filtrado Digital (IIR) y Sobremuestreo: Los datos brutos de los sensores son inherentemente ruidosos. Para obtener mediciones fiables, se aplican

dos técnicas de acondicionamiento de señal digital:

- Filtro IIR (Infinite Impulse Response): Es un filtro recursivo que suaviza la señal, reduciendo las fluctuaciones abruptas. El código lo implementa para estabilizar la lectura de presión.

- Sobremuestreo (Oversampling): Esta técnica consiste en tomar múltiples mediciones a alta velocidad y promediarlas. Esto reduce el ruido aleatorio y aumenta la resolución efectiva del sensor.

- Verificación de Integridad de Datos (Checksum[2]): La comunicación con sensores complejos como el LIDAR se realiza mediante tramas de datos. Para asegurar que los datos no se han corrompido durante la transmisión, se emplea una suma de verificación (checksum). El script de la Raspberry Pi valida cada trama sumando sus bytes y comparándolos con el byte de checksum enviado por el sensor. Si la validación falla, la trama se descarta, previniendo lecturas erróneas.

La funcionalidad final del vehículo rover se define por su capacidad de ser controlado a distancia (teleoperación) y de enviar datos a un operador que lo controla (telemetría).

- Transmisión de video en tiempo real (Streaming RTSP): Para un control efectivo más allá de la línea de visión, se implementa un sistema de vídeo en tiempo real. El script *camara.sh* utiliza el Protocolo de Transmisión en Tiempo Real (RTSP), para establecer y controlar sesiones de streaming de medios, permitiendo que un cliente (como FFMPEG[3] o MATLAB) se conecte al flujo de vídeo del rover.

- Telemetría y Registro de Datos Remoto: La telemetría se define como la recopilación de mediciones a distancia. El script *serial_a_ssh_luz.sh* implementa un sistema de telemetría robusto: Solicita datos al Arduino, recibe la respuesta y la redirige a través de un túnel seguro SSH para ser registrada directamente en el ordenador del operador.

- Análisis y Visualización de Datos (MATLAB): Finalmente, los datos brutos registrados en los archivos .txt son analizados y visualizados. Las aplicaciones de MATLAB App Designer (.mlapp) actúan como la herramienta de post-procesamiento, permitiendo la graficación, el análisis de tendencias y la interpretación científica de los fenómenos medidos por el rover.

[2]Checksum: Es un valor numérico generado a partir del contenido de un conjunto de datos, como un archivo o una transmisión de información.

[3]FFmpeg: Herramienta de línea de comandos para manipular y procesar archivos de audio y vídeo

4. Procedimiento (metodología de investigación)

El proceso a seguir en la elaboración del proyecto consta de las siguientes fases:

- Revisión Bibliográfica, principalmente estudio de aspectos de programación en C ++ enfocados a Arduino y MATLAB y aplicaciones de diseño automático y circuitos digitales.

- Estudio y trabajo en el diseño del vehículo para dar respuesta a los objetivos del proyecto. Estructura básica, complementos y módulos adicionales.

- Búsqueda de elementos y componentes del "Rover", así como estudio económico de viabilidad. Adquisición de material.

- Montaje y calibrado de los distintos módulos del vehículo. Integración e interconexión de los mismos sobre el chasis.

- Programación de las funcionalidades del vehículo, desde su movimiento a distancia hasta la conexión con la estación meteorológica o la cámara (toma de imágenes y vídeos), pasando por la geolocalización del mismo.

- Documentación de todo el proceso del estudio y trabajo realizado.

4.1. Revisión Bibliográfica

Para la realización de este proyecto se ha consultado literatura académica y proyectos de ingeniería que respaldan el diseño del vehículo rover en sus tres pilares: arquitectura de control, sensórica y telemática remota.

El conocimiento inicial de los principios de la física atmosférica y los conceptos fundamentales de presión, temperatura y humedad fue adquirido a través de la obra de García Ramos, Pérez Bernal, y Rodríguez Quintero, Problemas Resueltos de Meteorología [1]. Esta base conceptual es esencial para la correcta interpretación de los datos recolectados por el sensor BME-688 y la comprensión de los fenómenos que afectan la resistencia del sensor de gases.

La implementación de la lógica de control en la placa Arduino Mega (C++) y la programación de la interfaz remota en la Raspberry Pi (Python) se sustentan en una base sólida de conocimientos. El repaso y afianzamiento de conceptos de programación y algoritmos se realizó a través de los textos de Bernier Villamor, Cuaderno de Programación [2] y Problemas de Programación [3]. Estos recursos fueron cruciales para establecer la sintaxis y la estructura lógica necesaria para la comunicación serial y el control de actuadores.

El diseño de las aplicaciones de post-procesamiento y la interfaz de visualización en MATLAB App Designer se fundamenta en los conocimientos adquiridos a través de la obra varios autores, Programación y Métodos Numéricos para Ingeniería con MATLAB y Octave [4]. Este texto justificó la elección de MATLAB como la plataforma de análisis numérico para convertir los datos de telemetría en gráficos y resultados científicos.

La implementación de la arquitectura mecatrónica y los protocolos avanzados requirió una búsqueda complementaria de documentación técnica, ejemplos de código abierto y publicaciones de ingeniería que validan las soluciones finalmente adoptadas para este proyecto.

4.2. Búsqueda de componentes y recursos



Figura 1: Todos los componentes antes de montar.

Los componentes seleccionados para el montaje del vehículo rover han sido los siguientes:

- Chasis: SUNFOUNDER Galaxy RVR Mars Rover Kit. Piezas de aluminio para el chasis ya fabricadas.
- Componentes electrónicos:
  - SUNFOUNDER Galaxy RVR Mars Rover Kit, conjunto de sensores y módulos para el correcto funcionamiento del vehículo:
  - Arduino Shield: Para facilitar las conexiones y el control de motores con el arduino.
  - Servo utilizado para mover la cámara en vertical.
  - Sensor de ultrasonido utilizado para medir distancias y reconocer el terreno adyacente para poder anticiparse a colisiones.
  - Motores de CC y ruedas: seis motores de corriente continua controlados por la placa Arduino Shield, unidos a sus seis ruedas correspondientes para la movilidad.
  - Panel solar para alimentar con energía eléctrica la batería, con tiempo de carga completa de 7.2h (valor teórico), asumiendo una luz solar intensa, y emitiendo 6V y 660mA.
  - Batería modelo 18650 con una salida de 5V y 2A.

    - Tiras LED: dos tiras LEDs RGB neopixel de 4 unidades cada una para una iluminación del vehículo.

- Módulos y sensores adicionales:

    - BME 688: sensor de temperatura, humedad, presión y detección de gases.

    - Adafruit APDS-9960: sensor de proximidad, descomposición de luz y gestos.

    - Lidar youyeetoo TF-Luna: medidor de distancia láser.

    - Bobina de filamento de aluminio blanco para la impresión de distintas ampliaciones para el chasis.

    - Mini ordenador Raspberry Pi Zero 2WH:

    - Módulo de cámara RaspiCam de 5 megapíxeles y cable "flex" adaptado.

    - Iluminadores frontales creados con cuatro LEDs de alta potencia.

    - Microcontroladora Arduino Mega.

El cómputo general de todos los gastos en materiales y componentes es de 356,13 €, donde se tiene en cuenta la segunda compra de una placa Raspberry Pi, ya que la primera sufrió daños en una patilla por una soldadura defectuosa.

COMPONENTE	PRECIO
SUNFOUNDER Galaxy RVR Mars Rover Kit	125,00 €
BME 688	26,90 €
Gonnely Módulo Satélite GPS	13,19 €
GPS Antena SMA	15,88 €
Adafruit APDS 9960	16,53 €
Lidar youyeetoo TF-Luna	29,00 €
Bobina de filamento de aluminio blanco	18,99 €
Raspberry Pi Zero 2W	25,50 €
Raspberry Pi Zero 2WH	29,90 €
RaspiCam	19,99 €
Iluminadores frontales	12,00 €
Arduino Mega	23,25 €
TOTAL	356,13 €

Figura 2: Presupuesto general de elementos.

4.3. Montaje del vehículo y diseño de circuitos

Se procede con el montaje del chasis del vehículo, motores para la tracción mecánica, instalación de los sensores, módulo de control (microcontroladora + shield de Arduino) y sistema de alimentación eléctrica. Tras esto, se monta la cámara, los LEDs (inferiores como iluminadores frontales), sensores (BME-688, APDS-9960), GPS, LIDAR, y Raspberry Pi.

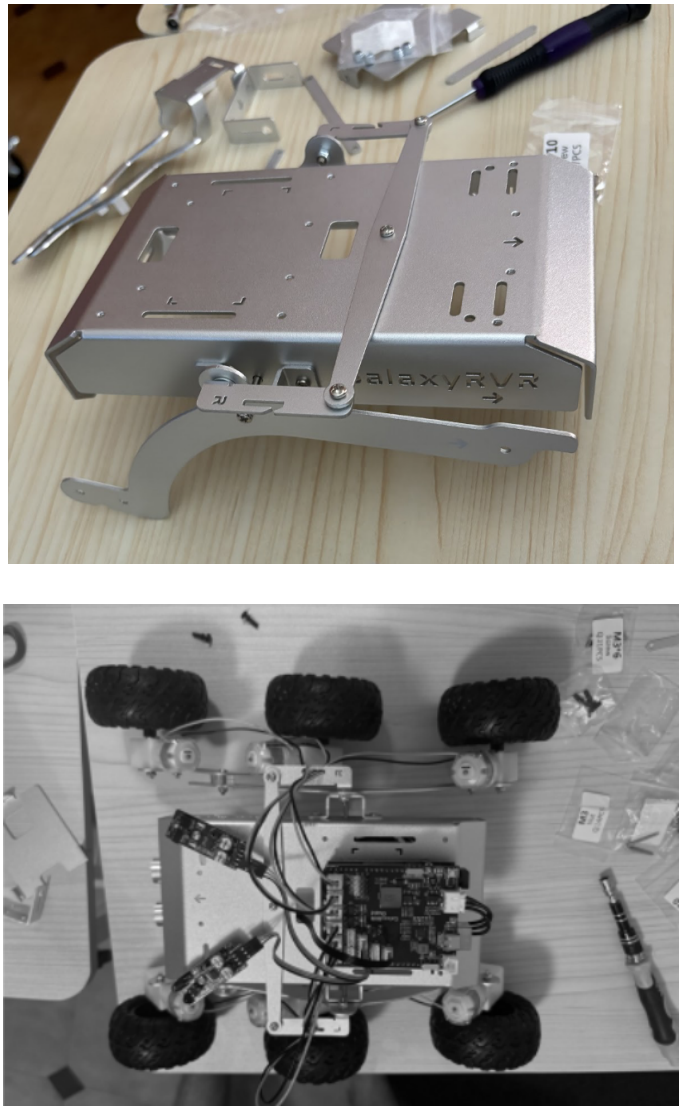


Figura 3: Montaje del vehículo.

Posteriormente, se realiza todo el conexionado y se comprueba su correcto funcionamiento para, posteriormente, empezar con la programación y hacer posible la comunicación entre los distintos componentes ya citados. Para llevar a cabo este proceso, primero se procede a la instalación del sistema operativo en la placa Raspberry Pi, en este caso RaspAP (Raspberry Access Point), un sistema operativo basado en Debian que crea un punto de acceso WiFi para poder conectarse a esta y acceder a los datos pertinentes. Posteriormente, se incorporó un cable USB desde la placa Arduino Mega hasta la placa Raspberry Pi para la comunicación entre ambos dispositivos.

Usando el software de diseño asistido Fritzing, se realizan los diseños y la simulación de los distintos circuitos eléctricos implementados en la construcción del vehículo rover.

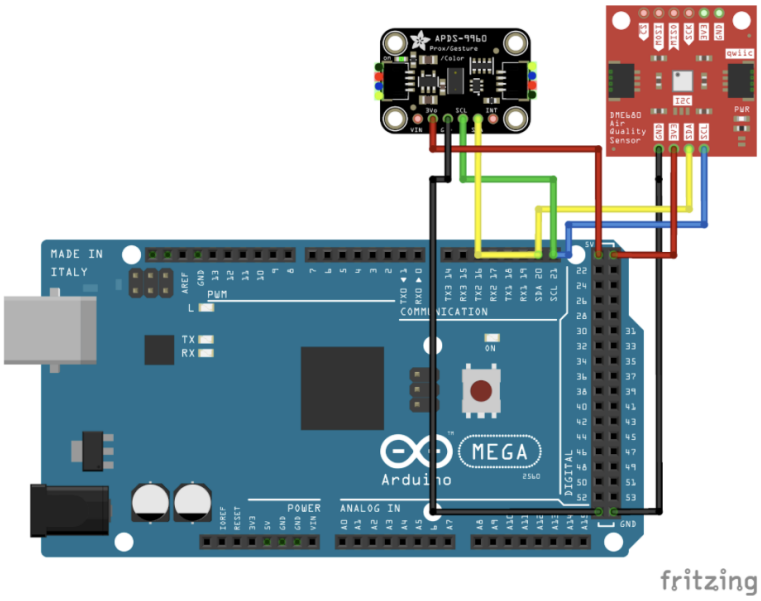


Figura 4: Esquema del circuito de los sensores APDS-9960 y BME-688. Ambos módulos están conectados a los pines SDA y SCL (protocolo I2C) de la placa Arduino Mega.

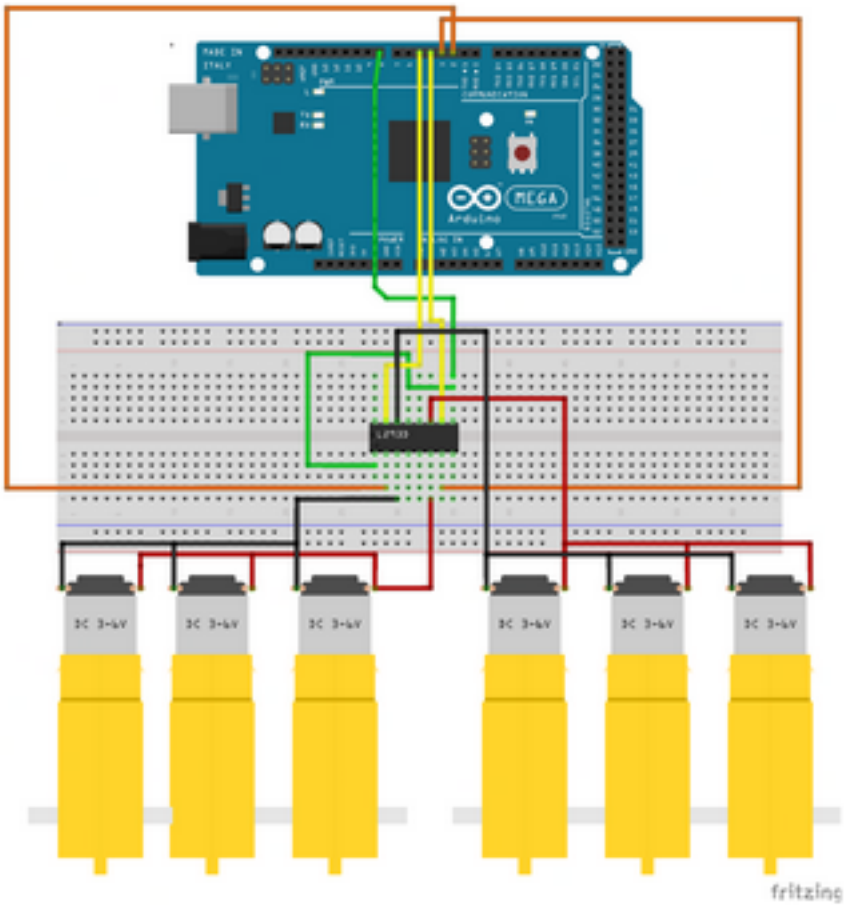


Figura 5: Esquema de la controladora de motores (circuito dentro de una shield), donde se usa un puente en H para un mejor y más eficiente control de estos. Los motores se controlan en dos grupos de tres motores:

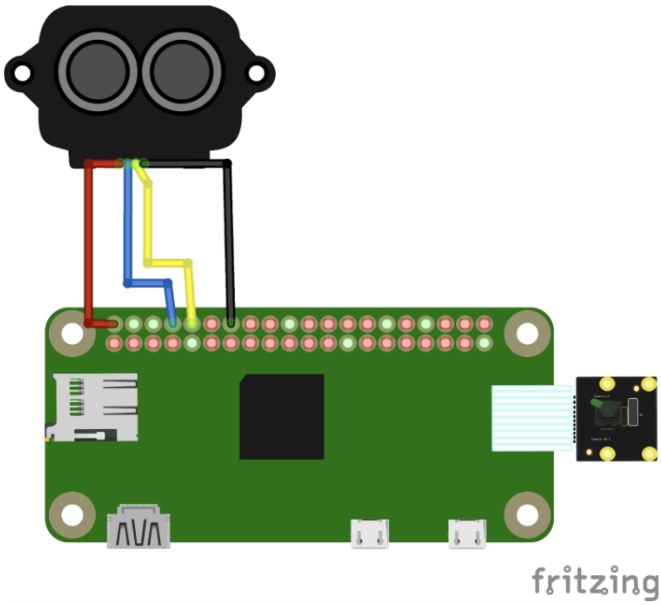


Figura 6: Esquema de la Raspberry Pi, con Lidar y cámara. La cámara está conectada a la placa mediante un cable específico plano y flexible de un metro y, el sensor Lidar, está conectado mediante protocolo RX-TX a la misma.

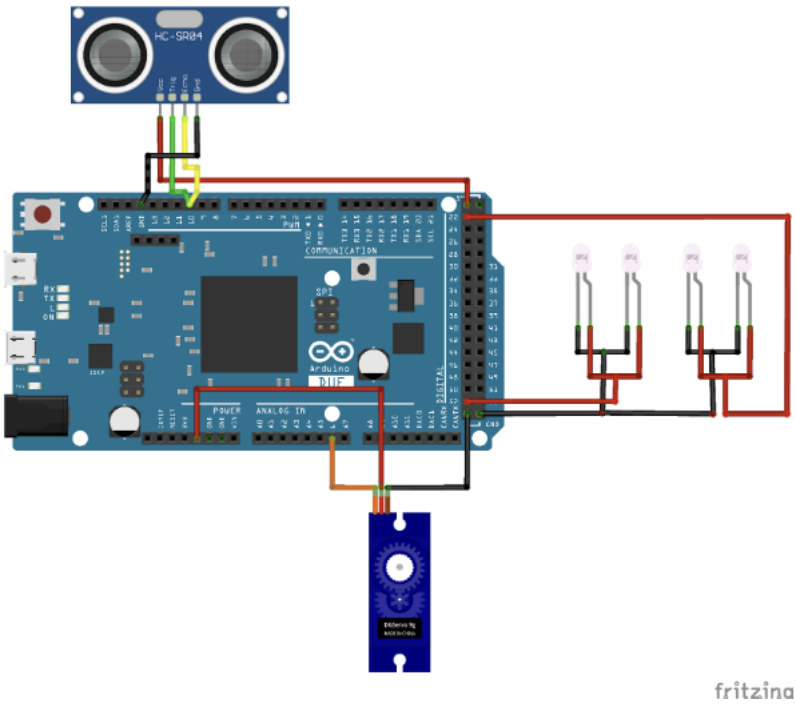


Figura 7: Esquema del servomotor, que se utiliza para dar movilidad a la cámara, conectado al pin analógico 6 (cable naranja). Además, en este esquema se puede ver la conexión de los LEDs frontales conectados a los pines digitales 52 y 23. Por otra parte, el sensor de ultrasonidos está conectado a los pines digitales 10 y 11 (pines trigger y echo).

4.4. Programación

El trabajo de programación de las placas (ordenador y microcontroladora) que controlan y administran los distintos sensores y actuadores del vehículo se puede dividir en distintas partes, claramente diferenciadas:

- Movimiento teledirigido y GUI[4].
- Estación meteorológica y de gases.
- Estación óptica.
- Video transmisión en directo.
- Comunicación a través de WiFi.
- MATLAB.

A continuación, se describe cómo funciona e interactúa el código con los distintos sensores y actuadores que componen el vehículo. El código, al completo, GitHub.

4.4.1. Movimiento teledirigido y GUI

La funcionalidad de movimiento teledirigido se establece mediante el puerto serial. La Raspberry Pi (ejecutando "arduino_inicio.sh") actúa recibiendo comandos desde MATLAB para enviarlos a la placa Arduino Mega, que se encarga del control físico de los motores y del resto de actuadores. El script motores de MATLAB es el programa que proporciona la interfaz de usuario y gestiona la comunicación serial con el vehículo.

- Interfaz gráfica: Se utiliza MATLAB App Designer para crear la Interfaz Gráfica de Usuario (GUI) del puesto de control. La GUI presenta los botones de dirección (Avance, Retroceso, Giro) y los botones para seleccionar la velocidad.
- Comunicación Serial Unificada: El principio clave de esta programación es la simplicidad y bajo "overhead" en la comunicación. Cada función de control se mapea a un único "byte" de comando:
  - La función avance() escribe el byte 'w' al Arduino.
  - La función stop() escribe '0'.
  - Las funciones de velocidad (v_100, v_66, v_33) envían bytes específicos ('3', '2', '1').

[4]GUI: Interfaz gráfica de usuario

- Esta estrategia de comando de un solo carácter minimiza la cantidad de datos transmitidos por el puerto serial, asegurando una respuesta de baja latencia en la teleoperación.

El código de Arduino es el responsable de decodificar el comando serial y traducirlo en control de potencia para los motores. El *loop* principal del Arduino verifica constantemente si hay datos en el puerto serial (*if (Serial.available() > 0)*). Al recibir el byte (variable comando), una estructura de control (*switch o if-else*) lo identifica y lleva a cabo la acción pertinente.

La gestión de los motores se realiza a través de la Modulación por Ancho de Pulso (PWM), esencial para variar la velocidad. Debido al número de motores y para evitar interferencias en su gestión, se emplea la librería *SoftPWM*. La función de movimiento recibe el valor de velocidad establecido por el operador. Internamente, utiliza *SoftPWMSet(PIN, v)* para ajustar el ciclo de trabajo. Un valor de v=255 corresponde al 100 % de potencia, mientras que valores menores (como v=66 ó v=33) reducen la potencia, logrando así los niveles de velocidad predefinidos.

El script *motores.mlapp* no solo gestiona el movimiento, sino que también sirve como la interfaz para controlar otros actuadores del vehículo y para iniciar rutinas de lectura de sensores en el Arduino. La interfaz permite al operador encender y apagar las luces auxiliares y controlar las tiras LED RGB montadas en el vehículo, enviando un comando simple para cada estado, además de recibir datos de los sensores LIDAR y ultrasonidos y mover el servo que porta la cámara, donde a través del script en la Raspberry envía por ssh el comando usando "*arduino_ envio.sh*" para que "*arduino_ inicio.sh*" reciba el nombre de la función a ejecutar.

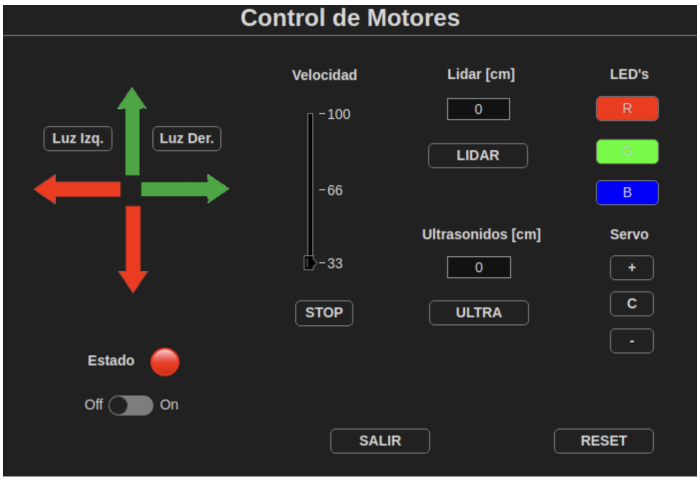


Figura 8: Ventana de control del sistema de motores del vehículo.

4.4.2. Estación meteorológica y de gases

En el script *serial_ bme.sh* ejecutado por la Raspberry, se encuentra toda la programación necesaria para interactuar con el sensor ambiental BME-688, el cual mide la temperatura y humedad del aire, la presión atmosférica y la Resistencia de Gases Volátiles Orgánicos (VOCs).

Antes de realizar cualquier lectura, el código de Arduino ejecuta una subrutina de configuración en la función setup() para optimizar el rendimiento del sensor, lo que garantiza mediciones estables y fiables.

El sobremuestreo es una técnica de procesamiento de señal utilizada para reducir el ruido y aumentar la resolución efectiva de las mediciones digitales. El código configura el sensor para que tome múltiples mediciones internas y las promedie antes de emitir un resultado final. Los niveles de sobremuestreo se ajustan en función de la criticidad de cada variable.

Se aplica un Filtro IIR (Respuesta Impulsional Infinita) para suavizar las fluctuaciones de los datos causadas por el ruido o cambios ambientales momentáneos. El filtro actúa como una medida que proporciona una telemetría mucho más estable y representativa de las condiciones ambientales.

La medición de gases requiere una temperatura específica del elemento sensor. El sistema configura el calentador de gas a 320 grados Celsius durante 150 milisegundos (la configuración recomendada para la mayoría de los VOCs), permitiendo que la resistencia del óxido metálico se estabilice antes de la lectura.

La ecuación barométrica

La presión que ejerce el aire sobre un cuerpo concreto se debe al peso de las capas de aire superiores:

$$dP = -\rho g dh$$

donde:

- P = presión atmosférica $[atm]$.
- ρ = densidad del aire $\left[\frac{kg}{m^3}\right]$.
- g = aceleración gravitatoria terrestre $\left[9,8\,\frac{m}{s^2}\right]$.
- h = altura barométrica $[m]$.

Ley de los gases ideales

Expresión ordinaria de la ley:

$$PV = nRT \qquad n = \frac{m}{M}$$

Luego:

$$PV = \frac{mRT}{M} \implies PM = \frac{m}{V}RT \implies PM = \rho RT$$

Si tomamos:

$$R' = \frac{R_{aire}}{M_{aire}} \quad ; \quad R_{aire} = 8,3145\,\frac{J}{mol \cdot K} \quad ; \quad M_{aire} = 0,0289644\,\frac{kg}{mol}$$

$$R' = \frac{R_{aire}}{M_{aire}} = \frac{8,3145}{0,0289644} \approx 287,06\,\frac{J}{kg \cdot K}$$

Obtenemos la expresión final:

$$P = \rho R'T \implies \rho = \frac{P}{R' \cdot T}$$

A continuación, insertamos el valor de la densidad en la ecuación barométrica:

$$dP = -\rho g dh = -\frac{P}{R' \cdot T} g dh = -\frac{Pg}{R' \cdot T} dh$$

Integramos a ambos lados de esta expresión (considerando un modelo isotermo):

$$\int_{P_0}^{P} dP = \int_{0}^{h} -\frac{Pg}{R' \cdot T} dh \implies \int_{P_0}^{P} dP = -\frac{Pg}{R' \cdot T} \int_{0}^{h} dh$$

$$\int_{P_0}^{P} \frac{dP}{P} = \frac{g}{R' \cdot T} \int_{h}^{0} dh \implies [ln(P)]_{P_0}^{P} = \frac{g}{R' \cdot T} [h]_{h}^{0}$$

$$(ln\,(P) - ln\,(P_0)) = \frac{g}{R' \cdot T}\,(0 - h) \implies ln\left(\frac{P}{P_0}\right) = -\frac{gh}{R' \cdot T}$$

Despejando la altura barométrica (h):

$$h = -\frac{R' \cdot T}{g} ln\left(\frac{P}{P_0}\right) = \frac{R' \cdot T}{g} ln\left(\frac{P_0}{P}\right)$$

Por lo que, finalmente, la expresión de la altura barométrica será:

$$h(P) = \frac{R' \cdot T}{g} ln\left(\frac{P_0}{P}\right)$$

El Arduino espera el comando de la Raspberry Pi para liberar la trama de datos del sensor. El loop principal de Arduino, espera recibir el comando serial 'e' (por 'estación') y, tras una lectura exitosa, el Arduino empaqueta los cinco parámetros (Temperatura, Humedad, Presión, Altitud Barométrica y Resistencia del Gas) en una única línea, separados por tabulaciones, y los envía al puerto serial para su posterior recepción.

4.4.3. Estación óptica

En el script *serial_luz.sh* ejecutado por la Raspberry, se encuentra toda la programación necesaria para interactuar con el sensor óptico APDS-9960, el cual mide la intensidad de la luz ambiental (Lux) y la información de color (RGB) del entorno.

El Arduino opera en un modo de lectura bajo demanda. El subsistema de la Raspberry Pi envía el comando serial 'l' (por 'luminosidad') para iniciar el ciclo de lectura. Una vez recibido el comando, el código garantiza la validez de los datos mediante un ciclo de espera activa, esta función pausa el código hasta que el sensor ha completado su ciclo de conversión analógico-digital y los datos de color están listos para ser leídos. La función devuelve cuatro valores sin procesar: las componentes de color Rojo (r), Verde (g), Azul (b) y la Componente Clara (c) (que se correlaciona con la intensidad de la luz. Estos datos se transmiten inmediatamente por el puerto serial al sistema de registro de la Raspberry, separados por tabulaciones. La Raspberry Pi es responsable de ejecutar el script serial_a_ssh_luz.sh para establecer un ciclo de adquisición continua y un registro estable de la telemetría óptica. El script funciona en un bucle que, tras una pausa predefinida, envía el comando 'l' al Arduino, lee la línea de datos resultante y la procesa para su almacenamiento.

Para garantizar que el flujo de datos no se interrumpa (evitando reinicios de la placa Arduino en cada nueva conexión), el script emplea el comando *exec 3<> "$puerto_serial"*. Este comando mantiene el puerto serial abierto de forma persistente, crucial para una recopilación de datos fiable a lo largo del tiempo.

Los valores brutos (V_{raw}) R, G, B y Lux leídos por el sensor requieren una etapa de post-procesamiento para convertirlos en valores físicos normalizados (V_{nor}) que sean útiles y comparables. Este paso se realiza en el sistema de análisis remoto. La fórmula utilizada aplica un factor de corrección y escalado al valor bruto:

$$V_{nor}\left(R, G, B, Lux\right) = \frac{V_{raw} \cdot 0,282}{FN}$$

El factor constante de 0,282 es un coeficiente de calibración empírico que ajusta la sensibilidad del sensor y convierte los valores brutos a una escala que representa magnitudes físicas (como Lux para la Componente Clara), asegurando la consistencia cuantitativa de las mediciones de color y luminosidad. En cuanto al factor de normalización (FN) es el divisor que permite convertir los conteo brutos del sensor APDS-9960 (valores R, G, B, C) en una escala de intensidad estandarizada. El sensor APDS-9960 proporciona sus lecturas como un valor entero que depende de dos parámetros críticos que se configuran en Arduino: el

tiempo de integración[5] y la ganancia[6]. El divisor 4 es el resultado de la ganancia y la resolución definidas en la configuración del sensor.

4.4.4. Localización RSSI

Para estimar la distancia lineal existente entre el rover y el ordenador de control se puede usar el parámetro RSSI (intensidad de la señal recibida) medida por el ordenador de control que está conectado a la red WiFi generada por dicho vehículo. Es una fórmula empírica y tiene sus limitaciones. También, se suele llamar a esta expresión modelo de pérdida de trayectoria logarítmica.

$$d = d^{\frac{RSSI_0 - RSSI}{10n}}$$

donde:

- d = distancia aproximada $[m]$.
- $RSSI_0$ = potencia recibida a 1 metro $[dBm]$[7]. Se debe de calibrar pero, normalmente a 1 metro, se recibe una intensidad de entre -30 y -45 dBm.
- RSSI = potencia medida $[dBm]$.
- n = índice de atenuación.
  - n = 2, para espacios libres o diáfanos.
  - n = 2,5 - 3,5, para interiores con paredes o con ciertas interferencias.
  - n = 4, para interiores muy cerrados o con muchas interferencias.

[5] Tiempo de integración: Es la duración exacta durante la cual el fotodiodo del sensor está activado para acumular fotones.

[6] Ganancia: Es el factor de amplificación electrónica aplicado a la señal analógica que sale del fotodiodo, justo antes de que el conversor Analógico-Digital (ADC) la digitalice.

[7] 1 dBm = 1 decibelio·mW

4.4.5. Video transmisión en directo

Para la gestión del flujo de vídeo, se han utilizado dos herramientas de código abierto que permiten el manejo de imagen a nivel profesional en entornos Linux:

- *RPi cam-vid*: Es la aplicación nativa de captura de vídeo para el sistema de cámara de Raspberry Pi. A diferencia de otras utilidades de streaming más pesadas, *rpicam-vid* accede directamente al hardware de codificación de la Raspberry Pi Zero 2W. Esto permite que el vídeo se comprima en formato H.264 en tiempo real sin saturar la CPU. En mi proyecto, esta herramienta es la encargada de capturar la imagen, aplicarle la rotación de 180° y enviarla como un flujo de datos puro a través de una dirección IP y un puerto específicos.

- *FFmpeg / FFplay*: *FFmpeg* es el marco multimedia líder que permite decodificar, codificar, transcodificar y reproducir prácticamente cualquier formato de vídeo. Dentro de este conjunto de herramientas, utilizo *ffplay*, que es un reproductor de vídeo muy ligero basado en las librerías de *FFmpeg*. Su gran ventaja en este proyecto es que permite un control total sobre los protocolos de red y los buffers. Mediante el uso de comandos específicos, puedo forzar a *ffplay* a que no almacene datos en memoria y muestre cada fotograma en el mismo instante en que llega, eliminando el retardo que tendrían los reproductores de vídeo convencionales.

4.4.6. Comunicación a través de WiFi

Toda la conectividad se establece a través de la red Wi-Fi utilizando el protocolo SSH (Secure Shell) para la mayoría de las transferencias. La característica más importante para la teleoperación es la comunicación de MATLAB con los scripts en bash de Raspberry, el proceso se inicia mediante la ejecución de *arduino_inicio.sh*, encargado de preparar un canal persistente a través de una tubería FIFO (Named Pipe) que garantiza la disponibilidad inmediata del puerto serie y evita el reinicio del Arduino Mega. Al activar cualquier comando desde la interfaz de MATLAB, la instrucción se transmite por red hacia la Raspberry Pi, donde es procesada por el script *arduino_envio.sh*. Este último se encarga de inyectar el carácter en la tubería FIFO o de ejecutar scripts específicos de Python para la obtención de datos de sensores críticos, como el LIDAR. Todo este flujo bidireccional asegura un control de motores instantáneo y una recepción de datos científicos ininterrumpida sin necesidad de conexión física.

Para el data logging de los sensores (BME-688 y APDS-9960), se utiliza un método de transferencia persistente y segura para escribir en un archivo en el puesto de control. Los scripts de shell (*serial_a_ssh.sh* y *serial_a_ssh_luz.sh*) están programados para usar la herramienta *sshpass*. Esta, permite la automatización del inicio de sesión SSH (usando credenciales definidas en el script) sin interacción manual, lo que es esencial para un proceso de registro en bucle. La lectura de telemetría, capturada del puerto serial por la Raspberry, se lleva directamente a través del túnel SSH al ordenador de control.

4.5. MATLAB

El software MATLAB se emplea para la fase post-procesamiento de los datos. Su capacidad para el cálculo vectorial y el diseño de interfaces gráficas permite transformar los archivos de texto de telemetría en datos científicos corregidos, normalizados y estadísticamente analizados.

El script *motores.mlapp* constituye el núcleo del control dinámico del Rover desde el puesto de mando. Su función principal es traducir las interacciones del operador en la interfaz gráfica en comandos binarios que son enviados a través de la red WiFi. La aplicación dispone de un panel con botones de dirección (Avance, Retroceso y Giros) que, al ser pulsados, ejecutan una función de *callback* encargada de realizar una conexión SSH con la Raspberry Pi para invocar al script *arduino_ envio.sh*. Dependiendo del botón accionado, la aplicación envía el carácter correspondiente ('w', 'a', 's', 'd' o '0' para detenerse), el cual se inyecta en la tubería del sistema para lograr una respuesta inmediata de los motores. Además, la GUI integra selectores para modificar los parámetros de velocidad en tiempo real y permite la lectura de retorno del sensor LIDAR y ultrasonidos; cuando se solicita una medición de distancia desde esta interfaz, la aplicación suspende momentáneamente el flujo serie para ejecutar el script de Python pertinente y mostrar la lectura numérica en el panel de control, garantizando así una navegación segura y supervisada.

El script *BME680.m* y *BME680.mlapp* son los responsables de aplicar correcciones y normalizaciones a los datos del sensor BME-688. La presión atmosférica medida se utiliza para calcular la altura barométrica. En ciertas condiciones, este cálculo puede generar valores numéricos negativos, lo cual es físicamente inconsistente en este contexto. El código aplica un filtro lógico para corregir esta anomalía.

Para facilitar la comparación con estándares científicos, el valor de Presión se normaliza dividiéndolo por la Presión Atmosférica Estándar[8], expresando el resultado en unidades de atmósferas.

$$P\,(atm) = \frac{P\,(Pa)}{101325}$$

El script *APDS-9960_ optica.m* toma los valores brutos del sensor óptico (R, G, B, C) y aplica la fórmula de calibración necesaria para su correcta interpretación.

Para que el operador pueda monitorizar los datos de forma intuitiva, se diseñaron interfaces de usuario dedicadas utilizando la herramienta MATLAB App Designer. Estas aplicaciones están programadas para cargar dinámicamente los

[8]Presión Atmosférica Estándar: 101325 Pa.

archivos de telemetría. Las aplicaciones permiten visualizar la telemetría histórica en gráficos de tendencia que se actualizan a medida que llegan nuevos datos. Esto, transforma una tabla de números en una herramienta de monitoreo del comportamiento del rover y su entorno a lo largo del tiempo.

El script *DISTANCIA_ WIFI.mlapp* es el módulo específico para la monitorización del RSSI (Received Signal Strength Indicator). Esta aplicación se encarga de ejecutar de forma cíclica comandos de red en la Raspberry Pi para obtener la potencia de la señal WiFi recibida en dBm. Los datos capturados son procesados en la estación de control para ofrecer una referencia visual sobre la calidad del enlace. Esta funcionalidad es crítica para la seguridad del Rover, ya que permite al operador identificar zonas de baja cobertura o interferencias que podrían comprometer la latencia del vídeo o la recepción de comandos de movimiento. El sistema vincula estos valores con los informes de telemetría, permitiendo correlacionar posibles pérdidas de datos con la degradación de la señal inalámbrica durante la misión de exploración.

5. Fuentes, materiales y métodos

5.1. Materiales

El sistema se basa en un control dual. La unidad maestra es la Raspberry Pi, la cual gestiona las comunicaciones Wi-Fi, la lógica de la interfaz gráfica y la transmisión de video. La unidad esclava es el Arduino Mega que se encarga de la ejecución de comandos en tiempo real y el control de los actuadores y sensores de bajo nivel.

La movilidad se garantiza mediante un conjunto de motores CC que son alimentados y controlados a través de un driver de motor de potencia, recibiendo comandos de Modulación por Ancho de Pulso (PWM) del Arduino.

La percepción ambiental se consigue gracias a múltiples sensores: el BME-688 funciona como estación meteorológica completa, capturando Temperatura, Humedad, Presión y Concentración de Gases Volátiles Orgánicos. El

sensor APDS-9960 es la estación óptica, midiendo la componente clara, el color (RGB) y la luminosidad. Finalmente, la detección de obstáculos se implementa con un sensor Ultrasónico para proximidad y un sensor LIDAR para medición de distancia de largo alcance y mayor fiabilidad.

5.2. Software

La programación se dividió según la función de cada componente. El control de bajo nivel en Arduino se desarrolló en lenguaje C ++, utilizando el IDE de Arduino. El software de los controladores de luces, LIDAR y ultrasonidos, fue desarrollado en Python 3 utilizando la librería *pyserial* para la comunicación serial con Arduino.

Los procesos de automatización y gestión de red se implementaron mediante SSH, en Bash, que son responsables de iniciar las rutinas de registro de información y el streaming de video. Finalmente, el post-procesamiento, el análisis estadístico y la visualización de toda la telemetría histórica se llevaron a cabo en MATLAB. Se utilizó la herramienta App Designer para construir interfaces gráficas dinámicas que cargan, analizan y representan los datos de los sensores en gráficos de tendencias para el operador.

5.3. Metodología

La programación de la movilidad se optimizó para la baja latencia mediante el envío de comandos seriales que pueden ir desde MATLAB, Bash, Python y por último Arduino, en otros casos saltando el paso de Python e incluso en orden inverso para los sensores. El control de potencia se logró mediante la técnica *SoftPWM*, esencial para simular la Modulación por Ancho de Pulso en múltiples pines digitales del Arduino y lograr el control lineal de la velocidad en tres niveles predefinidos.

En la adquisición de datos se prioriza la precisión. Para la estación meteorológica, se configuró el sobremuestreo y un Filtro IIR en el *setup* de Arduino para el BME-688, lo que reduce el ruido y estabiliza la telemetría. En la estación óptica, se implementó una espera activa[9] antes de cada lectura para asegurar que el sensor APDS-9960 hubiera completado su ciclo de tiempo de integración, garantizando la validez del dato de color. La fiabilidad del LIDAR se aseguró mediante una rutina de verificación por checksum programada en Python, que descarta cualquier trama de datos corrupta.

El control remoto gráfico se habilitó mediante MATLAB. Esto, permite que el script de la GUI se ejecute en el puesto de control unificando distintos scripts sin tener que entrar directamente a la Raspberry para su ejecución. Mientras la ventana de control se visualiza de forma segura en el ordenador remoto. El registro persistente de la telemetría se gestiona mediante el uso de túneles SSH automatizados con *sshpass*.

Un método vital para la estabilidad del sistema es el uso del comando *exec 3<> "$puerto_serial"* en los scripts *Bash*. Esta técnica mantiene el puerto serial USB abierto persistentemente, evitando el reinicio cíclico del Arduino Mega en cada lectura de la Raspberry y asegurando un flujo de datos continuo. La transmisión de video se optimizó para la baja latencia mediante el uso del protocolo *FFMPEG* en una tubería de datos entre el capturador y el publicador.

[9] Espera activa: Técnica donde un proceso repetidamente verifica una condición.

6. Resultados

6.1. MATLAB

6.1.1. Principal



Figura 9: Menú principal del programa.

El script *Principal.mlapp* constituye el núcleo operativo del sistema de control de la estación de tierra. Su función primordial es la centralización de los comandos de teleoperación y la monitorización básica del estado del vehículo en un entorno unificado.

6.1.2. Control de motores

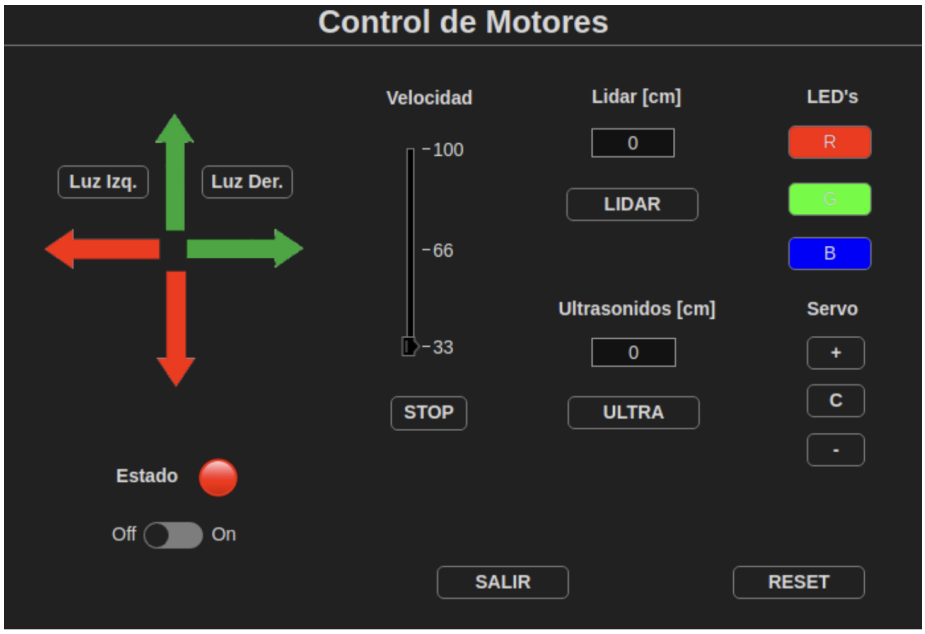


Figura 10: Ventana de control de motores.

Se ha conseguido una interfaz depurada y de rápida respuesta. Para que el sistema funcione correctamente, primero se debe accionar el interruptor de estado, poniéndose en verde cuando esté activo y así indicando la correcta conexión con el rover, pudiendo accionar cada una de las funcionalidades.

Las flechas para la dirección del rover, los botones de luz izquierda y luz derecha para los focos delanteros, el botón de stop para parar el rover, la barra de velocidad para ajustar la velocidad de las ruedas.

El botón de LIDAR indica la medida tomada por el LIDAR, ejecutando el script en Python y reenviándolo a MATLAB y el botón de ultrasonidos funcionando exactamente igual.

Los botones de R, G y B iluminan la tira led del rover y los botones del servo elevando, centrando o bajando la cámara. Finalmente, el botón de reset reinicia a los valores originales todo.

6.1.3. Sensor BME-688: Estación Meteorológica



Figura 11: Ventana principal de la estación meteorológica.

La aplicación *BME680.mlapp* ha sido diseñada para la monitorización en tiempo real y el análisis estadístico de las condiciones atmosféricas y la calidad del aire del entorno de exploración. A diferencia de la interfaz principal, este módulo se centra en la visualización científica de datos asíncronos.

El núcleo lógico de esta aplicación reside en la función *BME680_meteo.m*. La interfaz realiza una lectura cíclica del archivo de log *serial_bme.txt*, el cual es alimentado mediante el script de Bash *serial_bme.sh* desde la Raspberry Pi. Durante este proceso, se aplican las siguientes transformaciones matemáticas:

- Conversión de Presión: Los datos originales en Pascales (Pa) se normalizan a atmósferas (atm) dividiendo por la constante estándar de 101325.

- Algoritmo de Filtrado de Altitud: Se ha implementado un filtro lógico para corregir las derivas del sensor barométrico; si el cálculo de la altura relativa arroja valores negativos debido a fluctuaciones de presión, la aplicación los corrige automáticamente a 0 metros para mantener la coherencia física de la gráfica.

La GUI dispone de cuatro ejes de coordenadas donde se representan de forma dinámica las siguientes variables:

1. Temperatura y Humedad: Monitorización climática básica con actualización cada segundo.

2. Presión Atmosférica: Representación de la estabilidad del entorno.

3. Calidad del Aire (Gas): Medición de la resistencia del sensor de gases (kOhms), lo que permite detectar la presencia de compuestos orgánicos volátiles (VOC).

4. Altitud Barométrica: Estimación de la cota vertical del rover basada en la presión atmosférica.

Además de la monitorización en vivo, la aplicación integra una función de resumen estadístico (*BME680_ meteo_ est.m*). Mediante el procesado de toda la serie temporal acumulada en el archivo serial_bme.txt, la interfaz calcula y muestra:

- Valores mínimos, medios y máximos de cada magnitud.

- Marca de tiempo de la última actualización, garantizando la trazabilidad de los datos recolectados durante la misión.

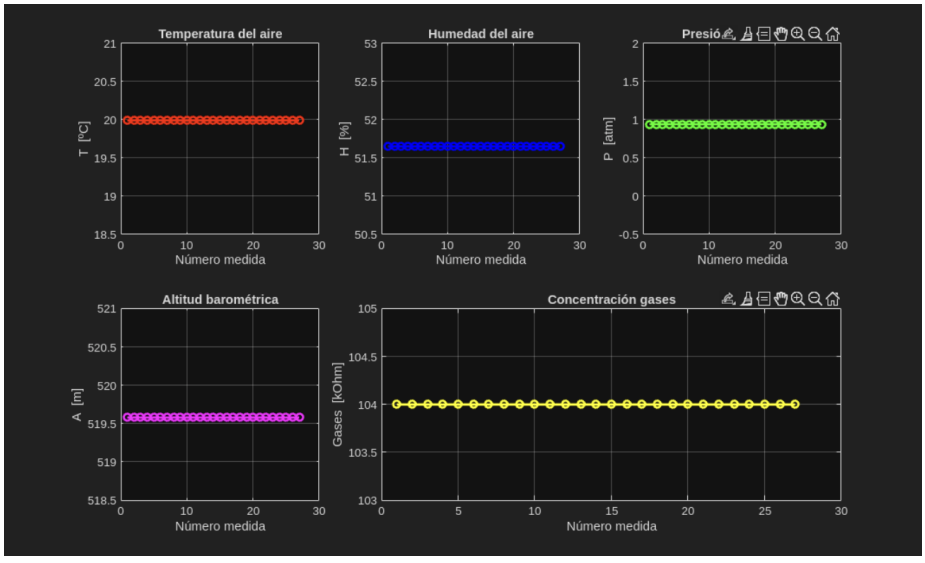


Figura 12: Ventana de gráficos en tiempo real.

6.1.4. Sensor APDS-9960: Estación Óptica



Figura 13: Ventana principal de la estación óptica.

La aplicación *APDS9960.mlapp* es la herramienta encargada de procesar e interpretar las condiciones lumínicas del entorno mediante la captura de datos del sensor digital de proximidad, luz ambiental y color APDS-9960.

A diferencia de un sensor de luz simple, esta interfaz desglosa la radiación electromagnética visible en sus componentes fundamentales. La lógica de la aplicación se apoya en la función *APDS9960_ optica.m*, que procesa los canales RGBC (Red, Green, Blue y Clear).

Un aspecto crítico en el desarrollo de esta aplicación ha sido la normalización de las lecturas. Dado que el sensor entrega valores digitales brutos, se ha aplicado un factor de escala de 0.282 en el código MATLAB para ajustar la sensibilidad y obtener valores comparables entre misiones.

La interfaz gráfica presenta los datos de forma intuitiva para el operador:

- Gráficas de Componentes (R, G, B): Tres ejes temporales permiten observar la predominancia de cada color. Esto es especialmente útil para identificar el tipo de iluminación artificial o la composición cromática de

superficies reflectantes.

- Intensidad Clear (Lux): Un indicador específico muestra la cantidad total de luz recibida, permitiendo evaluar la visibilidad para la navegación del rover.

- Histórico de Datos: Mediante la lectura del archivo serial_luz.txt, la aplicación genera un trazado histórico que permite analizar cambios bruscos en la iluminación, como la entrada en zonas de sombra o túneles.

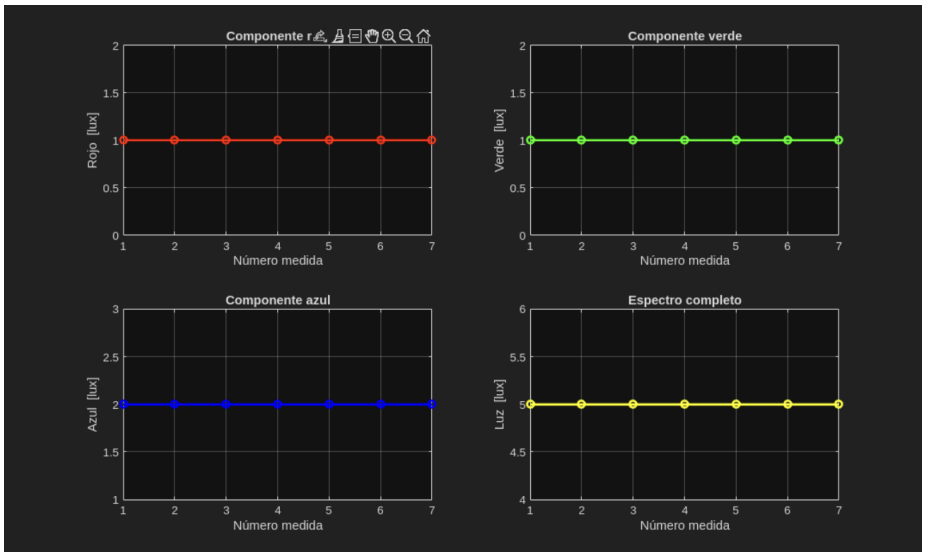


Figura 14: Ventana de gráficos en tiempo real.

La aplicación incluye un módulo de estadísticas (*APDS9960_optica_est.m*) que analiza el archivo de larga duración *serial_luz.txt*. Este módulo proporciona al investigador la media aritmética de cada canal cromático durante toda la misión y los picos máximos y mínimos de intensidad, fundamentales para el estudio de la respuesta del sensor ante cambios extremos de luz.

6.1.5. Distancia WiFi y cámara

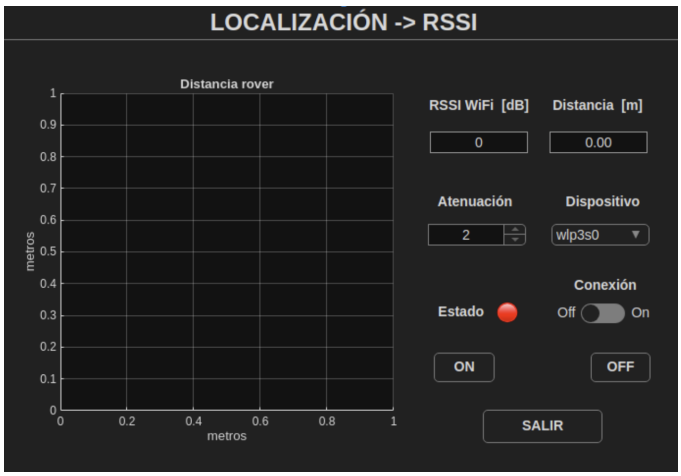


Figura 15: Ventana de localización mediante RSSI.

La aplicación *DISTANCIA_ WIFI.mlapp* implementa un sistema de localización estimada basado en la intensidad de la señal inalámbrica (RSSI). Este módulo permite suplir la carencia de sistemas GNSS en entornos interiores, proporcionando una métrica de proximidad entre el rover y el punto de acceso (AP).

La GUI permite al operador ajustar los parámetros de entorno en tiempo real para mejorar la precisión de la medida:

- Visualización del RSSI: Un medidor analógico y una gráfica temporal muestran la estabilidad de la conexión.
- Estimación de Distancia: Un indicador numérico traduce la pérdida de potencia en metros lineales.
- Ajuste Dinámico: La interfaz permite modificar el valor del coeficiente de atenuación n (típicamente, entre 2 para espacios abiertos y 4 para interiores con muros), permitiendo una calibración "in situ" según el escenario de exploración.

Para obtener el dato, MATLAB ejecuta de forma remota el comando de sistema *"iw dev wlan0 link"*. La aplicación utiliza expresiones regulares para extraer exclusivamente el valor numérico de la señal entre la cadena de texto devuelta por el sistema operativo, garantizando que el flujo de datos sea limpio y libre de caracteres no deseados.

6.2. Resultados

Se han realizado dos pruebas de campo completas con el vehículo rover en distintas localizaciones geográficas. La primera de ellas en la ciudad de Jaén, y la segunda, en el municipio de Almuñécar.

6.2.1. Prueba 1: Jaén (20/12/2025)

El tratamiento estadístico automatizado realizado por el programa arrojó los siguientes resultados para la estación meteorológica (sensor BME-680):

```
*******************************************************************************
***                          20-Dec-2025 15:38:31                           ***
*******************************************************************************
***                       ESTADÍSTICAS SENSOR BME-680                       ***

Temperatura aire:        mínimo = 10.98 ºC        media = 13.26 ºC      máximo = 19.99 ºC
Humedad relativa:        mínimo = 38.11 %         media = 45.69 %       máximo = 68.59 %
Presión atmosférica:     mínimo = 0.9352 atm      media = 0.9412 atm    máximo = 0.9428 atm
Altura barométrica:      mínimo = 494.22 m        media = 507.83 m         máximo = 561.16 m
Concentración gases:     mínimo = 24.00 kOhm      media = 184.26 kOhm   máximo = 292.00 kOhm

*******************************************************************************
```

Figura 16: Estadísticas estación meteorológica.

Y, para la estación óptica (APDS-9960):

```
*****************************************************************
***                      20-Dec-2025 15:46:57                 ***
*****************************************************************
***                 ESTADÍSTICAS SENSOR APDS-9960              ***

Componente roja:     mínimo = 1      media = 150.53        máximo = 1516
Componente verde:    mínimo = 1      media = 113.67        máximo = 1039
Componente azul:     mínimo = 2      media = 125.66        máximo = 1037
Luz completa:        mínimo = 5      media = 329.92        máximo = 1516

*****************************************************************
```

Figura 17: Estadísticas estación meteorológica.

6.2.2. Prueba 2: Almuñécar (22/12/2025)

Siguiendo el mismo proceder que en la primera prueba, el tratamiento estadístico automatizado realizado por el programa arrojó los siguientes resultados para la estación meteorológica (sensor BME-680):

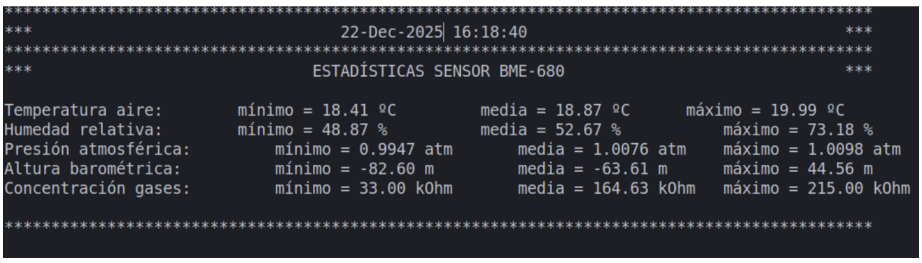


Figura 18: Estadísticas estación meteorológica.

Y, para la estación óptica (APDS-9960):

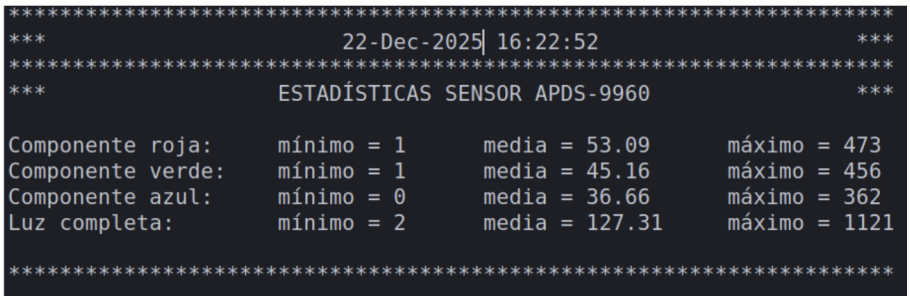


Figura 19: Estadísticas estación meteorológica.

6.3. RSSI

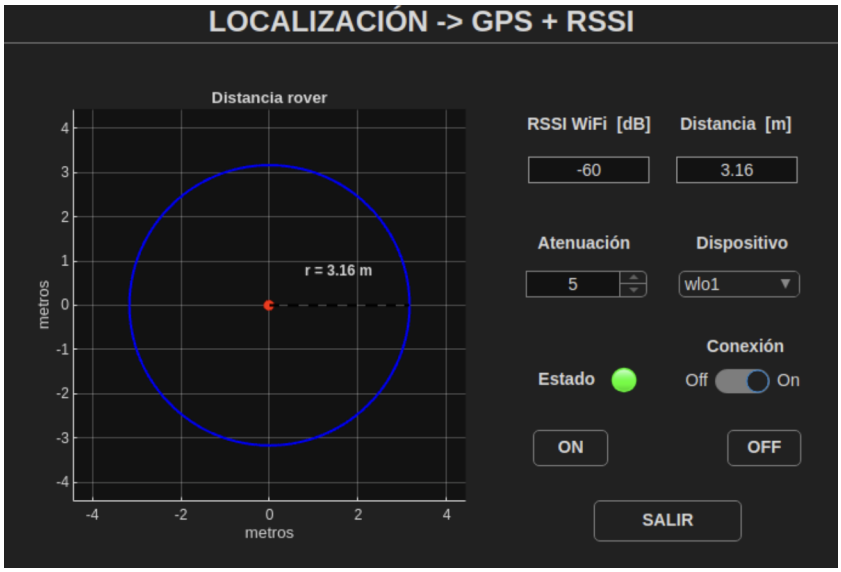


Figura 20: Vehículo rover localizado por RSSI.

Tras las pruebas de campo realizadas con el módulo de posicionamiento, se han extraído las siguientes conclusiones basadas en los datos observados en la interfaz de MATLAB:

1. Comportamiento de la Señal (Fading y Multipath): Como se observa en la captura de la aplicación , la señal RSSI no presenta un valor lineal constante, sino que sufre fluctuaciones de entre ± 3 y ± 5 dBm incluso con el rover en estado estacionario. Este fenómeno se identifica técnicamente como desvanecimiento rápido (Fast Fading), provocado por las reflexiones de la onda de 2.4 GHz en las paredes y el mobiliario del laboratorio (IES Santa Catalina de Alejandría). Para mitigar este ruido, el software implementa un refresco de lectura cada segundo, permitiendo obtener una tendencia visual clara a pesar de los picos puntuales.

2. Correlación Distancia-Potencia: Durante las pruebas, se ha validado que el modelo matemático responde correctamente a la inversa del cuadrado de la distancia:

- Zona de Proximidad ($<$ 2 metros): El RSSI se mantiene en valores superiores a -45 dBm, ofreciendo una estimación de distancia con un error inferior a 0.5 metros.

- Zona de Alejamiento ($>$ 5 metros): La potencia cae por debajo de los -65 dBm. En este rango, la sensibilidad aumenta, por lo que pequeños rebotes de la señal generan variaciones mayores en la distancia estimada (metros).

Mediante la calibración en tiempo real desde la App, se ha determinado que el exponente de pérdida de trayectoria ideal para el entorno de interiores del centro educativo es de n=5. Un valor superior al teórico en el vacío (n=2), lo cual es coherente debido a la presencia de obstáculos y la absorción de materiales.

6.4. Cámara



Figura 21: Imagen tomada por la cámara del vehículo rover.

En la captura de pantalla adjunta se observa el correcto funcionamiento del subsistema de visión del rover. A partir de esta prueba de rendimiento, se observa que gracias a la configuración del script *camara2.sh* en la Raspberry Pi y al uso de los flags de optimización en ffplay, se ha logrado un flujo de vídeo con una latencia inferior a los 200 ms. Esto es crítico para la teleoperación, ya que permite al operador reaccionar en tiempo real ante obstáculos detectados en el campo de visión. La imagen muestra una resolución de 1024x576 píxeles a 20 FPS, lo que supone un equilibrio óptimo entre nitidez y consumo de ancho de banda del canal WiFi.

Se observa una correcta reproducción cromática y una exposición equilibrada, facilitando la identificación de objetos incluso en las condiciones de iluminación del laboratorio. Se confirma que la ventana de reproducción se integra de forma coherente con el resto de la telemetría (RSSI, datos BME680, etc.), permitiendo una navegación "multimodal" donde el operador no solo depende de la vista, sino de los datos cuantitativos de los sensores que aparecen en los paneles laterales.

7. Problemas

7.1. Problemas físicos

El proceso de construcción y puesta en marcha del rover no estuvo exento de dificultades técnicas de diversa índole que requirieron una intervención analítica y práctica constante. Uno de los obstáculos más críticos y persistentes fue la gestión de la integridad física de las conexiones debido a las vibraciones mecánicas generadas por el tren motriz; estas vibraciones provocaron una fatiga prematura en las soldaduras de los componentes más sensibles, afectando de manera muy específica al sensor ambiental BME-688. Dado que este sensor utiliza el protocolo de comunicación I2C, cualquier micro-rotura en las soldaduras de los pines SDA o SCL derivaba en un colapso del bus de datos, generando lecturas erráticas o, en muchos casos, bloqueando la ejecución de los scripts en la Raspberry Pi al no recibir respuesta del esclavo.

A esta problemática de continuidad se sumó una incidencia de hardware mayor: la rotura de la unidad de procesamiento central, una Raspberry Pi Zero 2W. Durante las pruebas de prototipado de los pines GPIO, un cortocircuito accidental dejó la placa totalmente inoperativa, lo que supuso una interrupción crítica en el desarrollo del software de control y obligó no solo a la sustitución inmediata de la unidad, sino a un rediseño del aislamiento eléctrico y al uso de soportes mecánicos más robustos para proteger los componentes lógicos.

Finalmente, el sistema se enfrentó a notables insuficiencias energéticas durante las fases de carga máxima. Se observó que, al accionar simultáneamente los motores a pleno rendimiento junto con la transmisión de vídeo en alta definición, la demanda de corriente excedía la capacidad de entrega puntual de la batería, provocando caídas de tensión (denominadas técnicamente como brownouts) que resultaban en el reinicio inesperado de la controladora. Este último reto se solventó mediante la optimización de la arquitectura de alimentación, implementando reguladores de voltaje Step-Down independientes para separar la etapa de potencia de los motores de la etapa lógica de control, garantizando así una estabilidad operativa total y una telemetría fiable bajo cualquier condición de uso en el entorno de exploración.

7.2. Problemas de conexión

En el plano del software y las telecomunicaciones, el proyecto enfrentó desafíos complejos relacionados con la gestión de puertos y la estabilidad del enlace inalámbrico entre la estación de control y el vehículo. Uno de los problemas más críticos fue la gestión de los puertos lógicos para el streaming de vídeo; inicialmente, se produjeron conflictos de ocupación de sockets en la Raspberry Pi que impedían que el script *camara2.sh* liberara correctamente el puerto TCP tras una desconexión accidental o un cierre forzado. Esto provocaba que, en intentos de reconexión posteriores, el puerto apareciera como "en uso", inhabilitando la visión del rover hasta que se realizaba un reinicio manual de los servicios de red a través de la terminal.

Asimismo, la comunicación bidireccional entre MATLAB y la Raspberry Pi sufrió fallos sistemáticos debido a la inestabilidad inherente de la señal WiFi en el entorno de pruebas. Los túneles SSH establecidos para el intercambio de información se cerraban de forma prematura al detectar picos de latencia o pérdida de paquetes (*jitter*), lo que interrumpía abruptamente la actualización de los archivos .txt de telemetría. Esta desincronización era especialmente problemática en la interfaz de MATLAB, la cual, al intentar ejecutar la función *readmatrix* sobre archivos que estaban siendo escritos de forma incompleta o que habían quedado bloqueados por una sesión SSH caída, generaba excepciones críticas que congelaban la aplicación de control.

Para mitigar esta inestabilidad, fue necesario profundizar en la configuración avanzada del cliente *SSH*, implementando parámetros de persistencia como *ServerAliveInterval* y *ServerAliveCountMax* para forzar el mantenimiento del enlace. Además, se depuraron los scripts de *Bash* para asegurar que los procesos de escucha se reiniciaran automáticamente y se añadieron bloques de manejo de errores (*try-catch*) en el código de MATLAB. Estas medidas permitieron que el sistema fuera capaz de discriminar entre una pérdida temporal de señal y un fallo crítico del hardware, garantizando una navegación segura y una recepción de datos robusta incluso ante las interferencias electromagnéticas del laboratorio.

7.3. Problemas de impresión

Para optimizar la capacidad de muestreo del rover, se diseñó e imprimió en 3D una estructura auxiliar tipo mástil, encargada de sustentar los sensores BME-688 y APDS-9960 en una posición elevada. La fabricación de esta extensión presentó retos técnicos que afectaron a la estabilidad de la telemetría.

El problema más crítico fue el combadura de las piezas (warping) debido a la geometría esbelta del mástil. Al ser una pieza alta y estrecha, las fuerzas de contracción térmica del material tendían a despegar la base de la superficie de impresión, provocando una deformación que impedía que el mástil se mantuviera perfectamente vertical. Esto no solo afectaba a la estética del rover, sino que alteraba la orientación de los sensores, comprometiendo la precisión de las lecturas ópticas.

Asimismo, se experimentaron problemas de adhesión entre capas en la zona superior del soporte. Debido a las vibraciones sufridas durante la navegación, la falta de cohesión interna en algunos puntos provocó que el mástil se agrietara o llegara a romperse en la unión con los sensores. Para solventar estas deficiencias, se optó por una reorientación de la pieza en el laminador (slicer) para mejorar la resistencia mecánica y se aumentó la densidad del relleno (infill), logrando finalmente una extensión lo suficientemente robusta para mantener los sensores aislados de las interferencias térmicas y mecánicas del cuerpo principal del rover.

8. Conclusión

Se ha logrado con éxito el objetivo principal de diseñar, construir y programar un rover de exploración teleoperado. La investigación ha validado que la integración de la Raspberry Pi Zero 2 W como unidad de procesamiento local y MATLAB como estación de control terrestre permite una gestión de datos de alta precisión. Se ha alcanzado un hito importante al conseguir que sistemas heterogéneos (Python en el lado del servidor y MATLAB en el cliente) se comuniquen de forma bidireccional mediante protocolos SSH, garantizando que el operador disponga de una interfaz intuitiva y funcional para la toma de decisiones.

Una de las ideas principales que se desprenden de esta investigación es la importancia de la segmentación de datos sensoriales. El uso del sensor BME-688 no solo ha permitido medir variables meteorológicas, sino que ha abierto la puerta al análisis de la calidad del aire a través de compuestos orgánicos volátiles (VOC), mientras que el sensor APDS-9960 ha demostrado la viabilidad de realizar estudios espectrales de la luz sin necesidad de equipos de laboratorio costosos. La decisión estratégica de elevar estos sensores mediante un mástil impreso en 3D ha sido clave para evitar el ruido térmico de la electrónica interna, optimizando así la veracidad de los resultados obtenidos.

La superación de los retos documentados —desde las insuficiencias energéticas hasta los fallos de red y roturas físicas— constituye una de las ideas principales de este trabajo. La investigación concluye que en robótica móvil, la arquitectura de potencia debe ser independiente de la lógica para evitar caídas de tensión críticas. Asimismo, la gestión de errores en los túneles SSH ha demostrado que la estabilidad del software es tan vital como la integridad del hardware para el éxito de una misión de exploración.

Como cierre, se establece que este proyecto sienta las bases para futuras investigaciones en el campo de la robótica autónoma. La infraestructura actual permite la futura integración de algoritmos de inteligencia artificial para el reconocimiento de objetos mediante la cámara y el mapeo SLAM (Simultaneous Localization and Mapping) utilizando el sensor LIDAR, lo que supondría un salto cualitativo desde el control manual hacia la autonomía total del vehículo.

En conclusión, este trabajo de investigación ha permitido unir la ingeniería eléctrica, la fabricación aditiva y la programación de alto nivel, demostrando que es posible desarrollar soluciones tecnológicas complejas y eficaces para la exploración de entornos mediante el uso creativo y riguroso de la computación moderna.

9. Bibliografía

[1] J. E. García Ramos, F. Pérez Bernal, y J. Rodríguez Quintero, Problemas Resueltos de Meteorología. Huelva, España: Universidad de Huelva, 2005. ISBN: 978-84-96535-00-8. [En línea]. Disponible en: http://www.uhu.es/gem/docencia/meteo-ccaa/

Para el conocimiento de diversos conceptos de física atmosférica.

[2] J. L. Bernier Villamor y L. J. Herrera Maldonado, Cuaderno de Programación, 5a ed. Madrid, España: Editorial Técnica Avicam (Librería Fleming), 2024. ISBN: 978-84-10081-40-6.

[3] J. L. Bernier Villamor, Problemas de Programación, 4a ed. Madrid, España: Editorial Técnica Avicam (Librería Fleming), 2023. ISBN: 978-84-19494-69-6.

Para el repaso de distintos conocimientos sobre programación, en algunos de los lenguajes de programación.

[4] Varios autores, Programación y Métodos Numéricos para Ingeniería con MATLAB y Octave. Madrid, España: Ibergarceta Publicaciones S.L., 2021. ISBN: 978-84-19034-35-9.

Para el aprendizaje del uso y programación de MATLAB y sus aplicaciones.

10. Anexo

El código fuente desarrollado para este proyecto se encuentra disponible públicamente en GitHub y puede consultarse en el siguiente repositorio:

https://github.com/Roma-Software/Rover-TFB